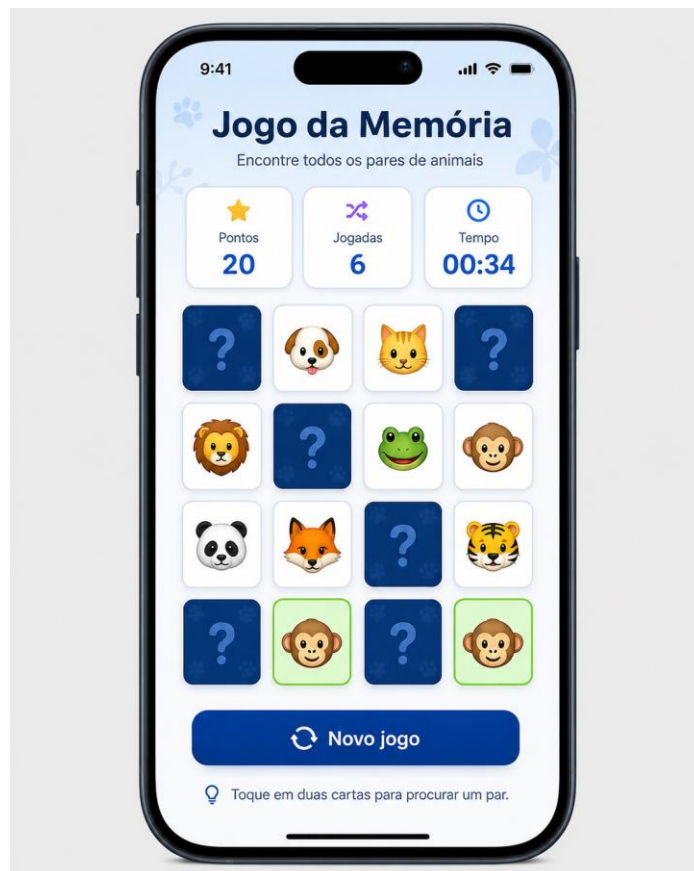


TUTORIAL PRÁTICO

Jogo da Memória com Emojis Animais

React Native, Expo SDK 54, Expo Go e Web

Guia passo a passo



Visual ilustrativo do aplicativo ao final da atividade.

Julho de 2026

Foco: uma única dificuldade, 16 cartas, placar, jogadas, cronômetro e vitória

Como utilizar este material

Este documento foi preparado como guia para conduzir uma aula prática com TypeScript, React ou React Native. As instruções indicam o que explicar, o que digitar, qual resultado esperar e como agir quando ocorrer um erro.

Objetivo do monitor

Conduzir a turma em pequenas etapas. Não peça aos alunos que copiem todo o código de uma vez. Em cada etapa, explique a ideia, digite junto com a turma, salve o arquivo e verifique o resultado.

Resultado esperado

- Aplicativo com 16 cartas, organizadas em quatro colunas.
- Oito pares de emojis de animais.
- Cartas fechadas com um ponto de interrogação.
- Comparação de duas cartas por jogada.
- Dez pontos por par encontrado, totalizando 80 pontos.
- Contador de jogadas completas.
- Cronômetro iniciado no primeiro toque e interrompido na vitória.
- Botão para iniciar uma nova partida.
- Janela de vitória com pontuação, jogadas e tempo.
- Execução no Expo Go e no navegador.

Parte	Conteúdo
0	Preparação do monitor e conceitos básicos.
1	Verificação e atualização do ambiente.
2	Criação do projeto Expo SDK 54.
3	Primeira execução no celular e na web.
4	Planejamento do jogo.
5	Configuração da navegação.
6	Construção passo a passo do aplicativo.
7	Testes orientados.
8	Solução de erros comuns.
9	Atividades de consolidação.
Apêndices	Código completo para conferência.

Parte 0 - Preparação do monitor

0.1 O que o aluno precisa saber antes de começar?

Não é necessário que o aluno saiba React Native. Contudo, o monitor deve apresentar rapidamente os conceitos abaixo antes da prática.

Termo	Explicação simples
Node.js	Programa que permite executar ferramentas JavaScript no computador.
npm	Gerenciador que instala bibliotecas e ferramentas usadas pelo projeto.
React Native	Tecnologia para criar interfaces de aplicativos usando JavaScript ou TypeScript.
Expo	Conjunto de ferramentas que facilita criar, executar e testar projetos React Native.
Expo Go	Aplicativo instalado no celular para abrir o projeto durante a aprendizagem.
TypeScript	JavaScript com tipos, usado para detectar erros e deixar o código mais claro.

Fala sugerida

“Hoje não vamos decorar comandos. Vamos aprender um fluxo: criar o projeto, abrir os arquivos, alterar o código, salvar e observar a mudança no celular ou no navegador.”

0.2 Conhecimentos praticados

- Componente: uma parte da interface, como uma tela, botão ou carta.
- Estado: uma informação que muda durante o uso, como pontos e tempo.
- Evento: uma ação do usuário, como tocar em uma carta.
- Array: uma lista; neste projeto, a lista de cartas.
- Condição: uma decisão do programa, como verificar se dois emojis são iguais.
- Renderização: o processo de apresentar os dados na tela.

0.3 Checklist do monitor antes da aula

- ☐ Testar o tutorial em um computador semelhante ao usado pela turma.
- ☐ Confirmar acesso à internet para instalar o projeto e as dependências.
- ☐ Confirmar que o Node.js LTS está instalado.
- ☐ Confirmar que o npm não está na versão 12 enquanto a incompatibilidade descrita neste material persistir.
- ☐ Instalar ou disponibilizar o Visual Studio Code.
- ☐ Pedir aos alunos que instalem o Expo Go antes da aula.
- ☐ Confirmar que computador e celular podem usar a mesma rede Wi-Fi.
- ☐ Separar uma alternativa: navegador web, caso o celular não consiga abrir o QR Code.
- ☐ Ter uma cópia do código completo disponível para comparação.
- ☐ Projetar o terminal e aumentar o tamanho da fonte para facilitar a leitura.

Parte 1 - Verificação e atualização do ambiente

Nota de compatibilidade

Este material usa explicitamente o Expo SDK 54 para compatibilidade com o Expo Go em dispositivo físico e também permite executar o mesmo projeto na web.

1 Abrir o terminal

No Windows, o aluno pode abrir o PowerShell. No macOS, pode abrir o Terminal. No Linux, pode usar o terminal padrão da distribuição.

Explique antes de digitar

O terminal é uma janela em que damos instruções ao computador por meio de comandos. Cada comando deve ser digitado e confirmado com a tecla Enter.

2 Verificar Node.js e npm

Digite os dois comandos, um de cada vez:

```
node --version  
npm --version
```

O primeiro comando mostra a versão do Node.js. O segundo mostra a versão do npm. Para o Expo SDK 54, utilize Node.js LTS com versão mínima 20.19.x. Uma versão LTS mais nova também pode ser usada, desde que o npm seja compatível com o criador do projeto.

Resultado esperado

Os comandos devem apresentar números de versão. Exemplo: Node.js v22.x e npm 10.x ou 11.x.

Comando não reconhecido

Se aparecer “node não é reconhecido”, “command not found” ou mensagem semelhante, o Node.js não está instalado ou o terminal ainda não reconheceu a instalação. Instale a versão LTS, feche todas as janelas do terminal, abra novamente e repita os comandos.

3 Evitar a incompatibilidade do npm 12

Há um problema conhecido em que determinadas versões do create-expo-app não conseguem interpretar a saída do comando npm pack quando o computador utiliza npm 12. O erro apresenta a mensagem “Could not parse JSON returned from npm pack”.

Se o comando npm --version começar com 12, instale temporariamente a versão 11:

```
npm install --global npm@11  
npm --version
```

Feche e abra o terminal após a instalação. O resultado do segundo comando deve começar com 11.

Importante

Não use sudo automaticamente e não altere permissões do sistema sem necessidade. Em computadores institucionais, solicite apoio do setor responsável caso a instalação global seja bloqueada.

4 Verificar o acesso ao registro do npm

```
npm config get registry
```

O resultado esperado é:

```
https://registry.npmjs.org/
```

Se aparecer outro endereço e a instituição não utiliza um registro próprio, configure o registro oficial:

```
npm config set registry https://registry.npmjs.org/  
npm ping
```

5 Verificar o criador de projetos

```
npx create-expo-app@latest --help
```

Na primeira execução, o npm pode perguntar se deseja instalar o pacote create-expo-app. Digite y e pressione Enter. O objetivo deste passo é apenas verificar se a ferramenta abre a tela de ajuda.

Não instale o antigo expo-cli global

Use os comandos iniciados por npx. O Expo CLI atual é instalado dentro do projeto com o pacote expo.

1.1 Diagnóstico rápido antes de prosseguir

- ☐ node --version apresenta um número.
- ☐ npm --version apresenta versão 10 ou 11.
- ☐ npm config get registry apresenta o registro correto.
- ☐ npx create-expo-app@latest --help funciona.
- ☐ O aluno sabe em qual pasta deseja criar o projeto.

Parte 2 - Criação do projeto com Expo SDK 54

1 Escolher uma pasta de trabalho

Peça aos alunos que criem uma pasta simples para atividades. Evite nomes com muitos espaços, acentos ou símbolos. Um exemplo adequado é “projetos”.

No terminal, navegue até a pasta desejada. Exemplo no Windows, caso a pasta esteja na Área de Trabalho:

Windows PowerShell

```
cd $HOME\Desktop\projetos
```

Exemplo no macOS ou Linux:

macOS ou Linux

```
cd ~/Desktop/projetos
```

Para iniciantes

Se a turma tiver dificuldade com o comando `cd`, o monitor pode abrir a pasta no Explorador/Finder e usar a opção “Abrir no terminal”, quando disponível.

2 Criar o projeto explicitamente com SDK 54

```
npx create-expo-app@latest jogo-da-memoria --template default@sdk-54
```

O comando faz quatro coisas:

- Cria uma pasta chamada `jogo-da-memoria`.
- Baixa o template padrão do Expo SDK 54.
- Configura TypeScript e Expo Router.
- Instala as dependências necessárias para Android, iOS e web.

Aguarde a conclusão

A instalação pode demorar alguns minutos. Não feche o terminal enquanto os pacotes estiverem sendo baixados.

3 Entrar na pasta do projeto

```
cd jogo-da-memoria
```

A partir deste ponto, todos os comandos do projeto devem ser executados dentro dessa pasta.

4 Abrir no Visual Studio Code

```
code .
```

O ponto significa “a pasta atual”. Se o comando `code` não funcionar, abra o Visual Studio Code manualmente e escolha Arquivo > Abrir Pasta > `jogo-da-memoria`.

5 Remover o exemplo inicial do template

```
npm run reset-project
```

O script move os arquivos de demonstração para uma pasta de exemplo e deixa a pasta app com os dois arquivos usados neste tutorial:

```
app/  
└─ _layout.tsx  
   index.tsx
```

Explique a estrutura

A pasta app contém as telas. O arquivo index.tsx será a tela principal. O arquivo _layout.tsx controla como as telas são organizadas.

6 Verificar a versão instalada

```
npm list expo react react-native
```

Na saída, a versão do pacote expo deve começar com 54. Os números completos podem variar conforme as correções publicadas para o SDK.

7 Verificar a saúde do projeto

```
npx expo install --check  
npx expo-doctor@latest
```

O primeiro comando verifica se as bibliotecas estão nas versões compatíveis. O segundo analisa problemas comuns do projeto.

Se o Expo indicar dependências incompatíveis, execute:

```
npx expo install --fix  
npx expo-doctor@latest
```

Evite npm update

Não use npm update para atualizar tudo indiscriminadamente. Em projetos React Native, as versões de algumas bibliotecas precisam permanecer alinhadas ao SDK.

Parte 3 - Primeira execução no celular e na web

1 Iniciar o servidor de desenvolvimento

```
npx expo start --clear
```

A opção `--clear` limpa o cache do Metro Bundler. Na primeira aula, isso reduz a chance de o aluno visualizar arquivos antigos ou receber erros de cache.

2 Abrir no Expo Go

1. Confirme que o Expo Go está instalado no celular.
2. Confirme que o computador e o celular estão na mesma rede Wi-Fi.
3. No Android, abra o Expo Go e use a opção de ler o QR Code.
4. No iPhone, use a câmera do sistema para ler o QR Code.
5. Aguarde o carregamento do projeto.

3 Abrir no navegador

Com o servidor aberto, clique no terminal e pressione a tecla:

```
w
```

O navegador deve abrir automaticamente. Essa opção é útil quando a rede bloqueia a comunicação com o celular.

4 Parar e reiniciar

Para encerrar o servidor, pressione:

```
Ctrl + C
```

Para iniciar novamente:

```
npx expo start --clear
```

Pausa pedagógica

Antes de escrever o jogo, peça aos alunos que alterem um texto simples no arquivo `app/index.tsx`, salvem e observem a atualização. Isso demonstra o ciclo `editar > salvar > visualizar`.

Parte 4 - Planejamento do Jogo da Memória

4.1 Regras do jogo

- O tabuleiro possui 16 cartas e 8 pares.
- O jogador toca em uma carta para revelar o emoji.
- Depois toca em uma segunda carta.
- Se os emojis forem iguais, o par permanece aberto e vale 10 pontos.
- Se forem diferentes, as cartas fecham após um pequeno intervalo.
- Cada escolha de duas cartas conta como uma jogada.
- O cronômetro começa no primeiro toque.
- O jogo termina quando os 8 pares forem encontrados.

4.2 Informações que mudam durante o jogo

Informação	Por que precisa ser armazenada?
Cartas	Para saber quais estão abertas e quais pares foram encontrados.
Primeira carta	Para comparar com a segunda carta.
Tabuleiro bloqueado	Para impedir o toque em uma terceira carta durante a comparação.
Pontos	Para mostrar o placar.
Jogadas	Para contar quantas comparações foram feitas.
Segundos	Para mostrar a duração da partida.
Cronômetro ativo	Para iniciar e parar a contagem.
Janela de vitória	Para mostrar o resultado final.

4.3 Formato de uma carta

```
type MemoryCard = {
  id: number;
  emoji: string;
  isFlipped: boolean;
  isMatched: boolean;
};
```

Propriedade	Significado
id	Identificador único da carta.
emoji	Animal apresentado quando a carta está aberta.
isFlipped	Indica se a carta está virada para cima.
isMatched	Indica se o par já foi encontrado.

Parte 5 - Configuração da tela

1 Editar app/_layout.tsx

Abra o arquivo app/_layout.tsx, apague o conteúdo e coloque:

```
import { Stack } from 'expo-router';

export default function RootLayout() {
  return (
    <Stack
      screenOptions={{
        headerShown: false,
      }}
    />
  );
}
```

O Stack organiza as telas do aplicativo. A opção headerShown: false remove o cabeçalho automático para que a interface do jogo use todo o espaço disponível.

Verificação

Salve o arquivo. Se aparecer um erro, confira parênteses, chaves, aspas e ponto e vírgula.

Parte 6 - Construção do aplicativo

Estratégia de aula

Nesta parte, construa o arquivo `app/index.tsx` em blocos. Depois de cada bloco, pare para explicar o objetivo. O código completo está no apêndice para conferência.

6.1 Preparar o arquivo e os imports

Abra `app/index.tsx`, apague o conteúdo e comece com os imports:

```
import { useEffect, useState } from 'react';
import {
  FlatList,
  Modal,
  Pressable,
  ScrollView,
  StyleSheet,
  Text,
  useWindowDimensions,
  View,
} from 'react-native';
import { StatusBar } from 'expo-status-bar';
```

- `useState`: guarda valores que mudam.
- `useEffect`: controla o cronômetro.
- `FlatList`: mostra a lista de cartas em quatro colunas.
- `Pressable`: recebe o toque do jogador.
- `Modal`: apresenta a mensagem de vitória.
- `ScrollView`: permite rolar a tela em dispositivos pequenos.
- `StyleSheet`: organiza os estilos.
- `useWindowDimensions`: calcula o tamanho das cartas de acordo com a tela.

6.2 Criar o tipo da carta

```
type MemoryCard = {
  id: number;
  emoji: string;
  isFlipped: boolean;
  isMatched: boolean;
};
```

O TypeScript usa esse tipo para verificar se toda carta possui as quatro informações necessárias.

6.3 Definir emojis e constantes

```
const EMOJIS = [' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' '];
const POINTS_PER_PAIR = 10;
const MAX_SCORE = EMOJIS.length * POINTS_PER_PAIR;
const NUMBER_OF_COLUMNS = 4;
const CARD_GAP = 10;
const HORIZONTAL_PADDING = 20;
```

A lista possui oito animais. Como cada animal será duplicado, o jogo terá 16 cartas. `MAX_SCORE` será 8×10 , ou seja, 80 pontos.

6.4 Criar e embaralhar o baralho

```
function createDeck(): MemoryCard[] {
  const duplicatedEmojis = [...EMOJIS, ...EMOJIS];

  const deck: MemoryCard[] = duplicatedEmojis.map((emoji, index) => ({
    id: index,
    emoji,
    isFlipped: false,
    isMatched: false,
  }));

  for (let index = deck.length - 1; index > 0; index--) {
    const randomIndex = Math.floor(Math.random() * (index + 1));
    const temporaryCard = deck[randomIndex];

    deck[index] = deck[randomIndex];
    deck[randomIndex] = temporaryCard;
  }

  return deck;
}
```

Primeiro, o operador ... cria uma nova lista contendo os emojis duas vezes. Depois, map transforma cada emoji em um objeto carta. Por fim, o laço embaralha as posições usando o algoritmo Fisher-Yates.

Demonstração simples

Escreva no quadro: $[2, 2] + [2, 2] = [2, 2, 2, 2]$. Depois explique que embaralhar muda somente a ordem, não a quantidade de cartas.

6.5 Formatar o cronômetro

```
function formatTime(totalSeconds: number): string {
  const minutes = Math.floor(totalSeconds / 60);
  const seconds = totalSeconds % 60;

  const formattedMinutes = String(minutes).padStart(2, '0');
  const formattedSeconds = String(seconds).padStart(2, '0');

  return `${formattedMinutes}:${formattedSeconds}`;
}
```

A função transforma segundos em uma apresentação no formato MM:SS. Por exemplo, 75 segundos tornam-se 01:15.

6.6 Criar o componente principal e calcular o tamanho das cartas

```
export default function Index() {
  const { width } = useWindowDimensions();

  const boardWidth = Math.min(
    width - HORIZONTAL_PADDING * 2,
    420,
  );

  const cardSize =
    (boardWidth - CARD_GAP * (NUMBER_OF_COLUMNS - 1)) /
    NUMBER_OF_COLUMNS;
```

A largura disponível é dividida entre quatro cartas. O tabuleiro fica limitado a 420 pixels para não ficar exageradamente largo no navegador.

6.7 Criar os estados do jogo

```
const [cards, setCards] = useState<MemoryCard[]>(() => createDeck());
const [selectedCardId, setSelectedCardId] = useState<number | null>(null);
const [boardLocked, setBoardLocked] = useState(false);
const [score, setScore] = useState(0);
const [attempts, setAttempts] = useState(0);
const [seconds, setSeconds] = useState(0);
const [timerRunning, setTimerRunning] = useState(false);
const [showVictory, setShowVictory] = useState(false);
```

A função `useState` devolve duas partes: o valor atual e uma função para alterar esse valor. Exemplo: `score` contém os pontos e `setScore` altera os pontos.

Pergunta para a turma

“Qual estado deve mudar quando o jogador encontra um par?” Resposta esperada: `cards` e `score`. Na última jogada, também `timerRunning` e `showVictory`.

6.8 Implementar o cronômetro

```
useEffect(() => {
  if (!timerRunning) {
    return;
  }

  const intervalId = setInterval(() => {
    setSeconds((currentSeconds) => currentSeconds + 1);
  }, 1000);

  return () => {
    clearInterval(intervalId);
  };
}, [timerRunning]);
```

Quando `timerRunning` é verdadeiro, `setInterval` aumenta os segundos a cada 1.000 milissegundos. A função de retorno remove o intervalo quando o cronômetro para ou o componente é desmontado.

6.9 Criar a função de novo jogo

```
function startNewGame() {
  setCards(createDeck());
  setSelectedCardId(null);
  setBoardLocked(false);
  setScore(0);
  setAttempts(0);
  setSeconds(0);
  setTimerRunning(false);
  setShowVictory(false);
}
```

A função restaura todos os estados e cria um novo baralho embaralhado.

6.10 Processar o toque em uma carta

Esta é a principal regra do jogo. Digite a função em quatro blocos.

Bloco A - impedir toques inválidos

```
function handleCardPress(card: MemoryCard) {
  if (boardLocked || card.isFlipped || card.isMatched) {
    return;
  }

  if (!timerRunning && score < MAX_SCORE) {
```

```
    setTimerRunning(true);
  }
}
```

O return encerra a função quando o tabuleiro está bloqueado, quando a carta já está aberta ou quando o par já foi encontrado. O cronômetro começa no primeiro toque válido.

Bloco B - virar a carta e guardar a primeira escolha

```
setCards((currentCards) =>
  currentCards.map((currentCard) =>
    currentCard.id === card.id
      ? { ...currentCard, isFlipped: true }
      : currentCard,
  ),
);

if (selectedCardId === null) {
  setSelectedCardId(card.id);
  return;
}
```

map cria uma nova lista. Apenas a carta tocada recebe isFlipped: true. Se ainda não havia primeira carta, o identificador é guardado e a função termina.

Bloco C - localizar a primeira carta e comparar

```
const firstCard = cards.find(
  (currentCard) => currentCard.id === selectedCardId,
);

if (!firstCard) {
  setSelectedCardId(null);
  return;
}

setBoardLocked(true);
setAttempts((currentAttempts) => currentAttempts + 1);

const cardsMatch = firstCard.emoji === card.emoji;
```

find procura a primeira carta pelo id. Ao selecionar a segunda carta, o tabuleiro é bloqueado e o contador de jogadas aumenta. A variável cardsMatch será verdadeira quando os emojis forem iguais.

Bloco D - tratar acerto e erro

```
if (cardsMatch) {
  setTimeout(() => {
    setCards((currentCards) =>
      currentCards.map((currentCard) => {
        const belongsToPair =
          currentCard.id === firstCard.id ||
          currentCard.id === card.id;

        if (belongsToPair) {
          return {
            ...currentCard,
            isFlipped: true,
            isMatched: true,
          };
        }
      })
    );

    return currentCard;
  }),
);

const nextScore = score + POINTS_PER_PAIR;

setScore(nextScore);
setSelectedCardId(null);
```

```

    setBoardLocked(false);

    if (nextScore === MAX_SCORE) {
      setTimerRunning(false);
      setShowVictory(true);
    }
  }, 450);
} else {
  setTimeout(() => {
    setCards((currentCards) => {
      currentCards.map((currentCard) => {
        const wasSelected =
          currentCard.id === firstCard.id ||
          currentCard.id === card.id;

        if (wasSelected && !currentCard.isMatched) {
          return { ...currentCard, isFlipped: false };
        }

        return currentCard;
      }),
    );

    setSelectedCardId(null);
    setBoardLocked(false);
  }, 800);
}
}

```

No acerto, as duas cartas recebem `isMatched: true` e o jogador ganha 10 pontos. No erro, as cartas voltam a `isFlipped: false` depois de 800 milissegundos. O atraso permite que o aluno veja a segunda carta antes de ela fechar.

6.11 Criar a interface

Agora adicione o `return` do componente. Ele possui cinco partes: cabeçalho, placar, tabuleiro, botão e modal.

```

return (
  <View style={styles.screen}>
    <StatusBar style="dark" />

    <ScrollView
      contentContainerStyle={styles.screenContent}
      showsVerticalScrollIndicator={false}
    >
      <View style={styles.header}>
        <Text style={styles.title}>Jogo da Memória</Text>
        <Text style={styles.subtitle}>
          Encontre todos os pares de animais
        </Text>
      </View>

      <View style={styles.scoreboard}>
        <View style={styles.scoreItem}>
          <Text style={styles.scoreLabel}>Pontos</Text>
          <Text style={styles.scoreValue}>{score}</Text>
        </View>

        <View style={styles.scoreItem}>
          <Text style={styles.scoreLabel}>Jogadas</Text>
          <Text style={styles.scoreValue}>{attempts}</Text>
        </View>

        <View style={styles.scoreItem}>
          <Text style={styles.scoreLabel}>Tempo</Text>
          <Text style={styles.scoreValue}>{formatTime(seconds)}</Text>
        </View>
      </View>

      <View style={[styles.board, { width: boardWidth }]}>
        <FlatList
          data={cards}
          numColumns={NUMBER_OF_COLUMNS}
          keyExtractor={(item) => String(item.id)}
        />
      </View>
    </ScrollView>
  </View>
)

```

```

    scrollEnabled={false}
    extraData={` ${boardLocked} - ${cardSize} `}
    columnWrapperStyle={styles.cardRow}
    contentContainerStyle={styles.boardContent}
    renderItem={({ item }) => {
      const isVisible = item.isFlipped || item.isMatched;

      return (
        <Pressable
          accessibilityRole="button"
          accessibilityLabel={
            isVisible
              ? `Carta com o emoji ${item.emoji}`
              : 'Carta virada para baixo'
          }
          disabled={boardLocked || item.isFlipped || item.isMatched}
          onPress={() => handleCardPress(item)}
          style={({ pressed }) => [
            styles.card,
            { width: cardSize, height: cardSize },
            isVisible ? styles.cardVisible : styles.cardHidden,
            item.isMatched && styles.cardMatched,
            pressed && styles.cardPressed,
          ]}
        >
          <Text
            style={[
              styles.cardText,
              !isVisible && styles.hiddenCardText,
            ]}
          >
            {isVisible ? item.emoji : '?'}
          </Text>
        </Pressable>
      );
    }}
  />
</View>

<Pressable
  accessibilityRole="button"
  disabled={boardLocked}
  onPress={startNewGame}
  style={({ pressed }) => [
    styles.restartButton,
    pressed && styles.restartButtonPressed,
    boardLocked && styles.restartButtonDisabled,
  ]}
>
  <Text style={styles.restartButtonText}>Novo jogo</Text>
</Pressable>

<Text style={styles.instructions}>
  Toque em duas cartas para procurar um par.
</Text>
</ScrollView>

<Modal
  visible={showVictory}
  transparent
  animationType="fade"
  onRequestClose={() => setShowVictory(false)}
>
  <View style={styles.modalOverlay}>
    <View style={styles.modalContent}>
      <Text style={styles.modalEmoji}> </Text>
      <Text style={styles.modalTitle}>Parabéns!</Text>
      <Text style={styles.modalText}>
        Você encontrou todos os pares.
      </Text>

      <View style={styles.finalResult}>
        <Text style={styles.finalResultText}>Pontuação: {score}</Text>
        <Text style={styles.finalResultText}>Jogadas: {attempts}</Text>
        <Text style={styles.finalResultText}>
          Tempo: {formatTime(seconds)}
        </Text>
      </View>
    </View>
  </View>

```



```

    </View>

    <Pressable
      accessibilityRole="button"
      onPress={startNewGame}
      style={({ pressed }) => [
        styles.modalButton,
        pressed && styles.restartButtonPressed,
      ]}
    >
      <Text style={styles.modalButtonText}>Jogar novamente</Text>
    </Pressable>
  </View>
</View>
</Modal>
</View>
);
}

```

A interface usa renderização condicional para decidir se mostra o emoji ou o ponto de interrogação. As propriedades de acessibilidade descrevem os botões e as cartas para tecnologias assistivas.

6.12 Adicionar os estilos

```

const styles = StyleSheet.create({
  screen: {
    flex: 1,
    backgroundColor: '#f3f4f6',
  },
  screenContent: {
    alignItems: 'center',
    paddingTop: 50,
    paddingBottom: 28,
    paddingHorizontal: HORIZONTAL_PADDING,
  },
  header: {
    alignItems: 'center',
    marginBottom: 20,
  },
  title: {
    color: '#172554',
    fontSize: 30,
    fontWeight: '700',
    textAlign: 'center',
  },
  subtitle: {
    color: '#64748b',
    fontSize: 15,
    marginTop: 5,
    textAlign: 'center',
  },
  scoreboard: {
    width: '100%',
    maxWidth: 420,
    flexDirection: 'row',
    justifyContent: 'space-between',
    marginBottom: 22,
  },
  scoreItem: {
    width: '31%',
    alignItems: 'center',
    backgroundColor: 'ffffff',
    borderColor: 'dbeafe',
    borderRadius: 14,
    borderWidth: 1,
    paddingHorizontal: 5,
    paddingVertical: 12,
  },
  scoreLabel: {
    color: '#64748b',
    fontSize: 13,
  },
  scoreValue: {
    color: '#1e3a8a',
    fontSize: 20,
  },
});

```

```

    fontWeight: '700',
    marginTop: 3,
  },
  board: {
    alignSelf: 'center',
  },
  boardContent: {
    gap: CARD_GAP,
  },
  cardRow: {
    justifyContent: 'space-between',
  },
  card: {
    alignItems: 'center',
    justifyContent: 'center',
    borderRadius: 16,
    borderWidth: 2,
  },
  cardHidden: {
    backgroundColor: '#1e3a8a',
    borderColor: '#1e40af',
  },
  cardVisible: {
    backgroundColor: 'ffffff',
    borderColor: '#93c5fd',
  },
  cardMatched: {
    backgroundColor: '#dcfce7',
    borderColor: '#22c55e',
  },
  cardPressed: {
    opacity: 0.75,
  },
  cardText: {
    fontSize: 34,
  },
  hiddenCardText: {
    color: 'ffffff',
    fontSize: 28,
    fontWeight: '700',
  },
  restartButton: {
    width: '100%',
    maxWidth: 420,
    alignItems: 'center',
    backgroundColor: '#1e3a8a',
    borderRadius: 14,
    marginTop: 20,
    paddingVertical: 14,
  },
  restartButtonPressed: {
    opacity: 0.8,
  },
  restartButtonDisabled: {
    opacity: 0.5,
  },
  restartButtonText: {
    color: 'ffffff',
    fontSize: 16,
    fontWeight: '700',
  },
  instructions: {
    color: '#64748b',
    fontSize: 14,
    marginTop: 14,
    textAlign: 'center',
  },
  modalOverlay: {
    flex: 1,
    alignItems: 'center',
    justifyContent: 'center',
    backgroundColor: 'rgba(15, 23, 42, 0.65)',
    padding: 24,
  },
  modalContent: {
    width: '100%',
    maxWidth: 360,

```

```

    alignItems: 'center',
    backgroundColor: '#ffffff',
    borderRadius: 24,
    padding: 28,
  },
  modalEmoji: {
    fontSize: 58,
  },
  modalTitle: {
    color: '#172554',
    fontSize: 28,
    fontWeight: '700',
    marginTop: 8,
  },
  modalText: {
    color: '#64748b',
    fontSize: 16,
    marginTop: 8,
    textAlign: 'center',
  },
  finalResult: {
    width: '100%',
    backgroundColor: '#eff6ff',
    borderRadius: 14,
    marginVertical: 20,
    padding: 16,
  },
  finalResultText: {
    color: '#1e3a8a',
    fontSize: 16,
    marginVertical: 3,
    textAlign: 'center',
  },
  modalButton: {
    width: '100%',
    alignItems: 'center',
    backgroundColor: '#1e3a8a',
    borderRadius: 14,
    paddingVertical: 14,
  },
  modalButtonText: {
    color: '#ffffff',
    fontSize: 16,
    fontWeight: '700',
  },
});

```

StyleSheet.create organiza os estilos em um único objeto. As cores azuis representam o estado normal, o branco representa cartas abertas e o verde destaca os pares encontrados.

6.13 Salvar e observar o resultado

1. Salve app/index.tsx.
2. Observe o terminal para verificar erros.
3. Aguarde a atualização automática no Expo Go ou no navegador.
4. Toque em uma carta e confirme se o cronômetro começa.
5. Encontre um par e confirme se o placar aumenta em 10 pontos.

Parte 7 - Roteiro de testes

Teste	Resultado esperado
Abrir o aplicativo	16 cartas fechadas, 0 pontos, 0 jogadas e 00:00.
Tocar na primeira carta	Emoji aparece e o cronômetro começa.
Tocar novamente na mesma carta	Nada acontece.

Teste	Resultado esperado
Escolher duas cartas diferentes	Jogadas aumenta em 1; as cartas fecham após breve espera.
Escolher duas cartas iguais	As cartas ficam verdes e o placar aumenta em 10.
Tocar em um par encontrado	Nada acontece.
Clicar em Novo jogo	Baralho, pontos, jogadas e tempo são reiniciados.
Encontrar todos os pares	Cronômetro para e a janela de vitória aparece.
Abrir no navegador	O layout permanece centralizado e responsivo.

7.1 Técnica de teste para o monitor

Como as cartas são embaralhadas, encontrar todos os pares pode levar tempo. Para testar rapidamente o modal de vitória, o monitor pode temporariamente substituir a lista de emojis por apenas dois animais.

```
const EMOJIS = [ ' ', ' ' ];
```

Não deixar no projeto final

Depois de testar, restaure a lista completa com oito emojis para voltar às 16 cartas.

Parte 8 - Solução de erros comuns

8.1 Erro ao criar o projeto: Could not parse JSON returned from npm pack

Causa provável

O npm 12 alterou o formato da resposta usada por versões do create-expo-app. Enquanto a ferramenta não estiver ajustada no ambiente utilizado, use npm 11.

```
node --version
npm --version
npm install --global npm@11
```

Feche o terminal, abra novamente e confirme:

```
npm --version
```

Exclua a pasta incompleta antes de tentar novamente.

Windows PowerShell

```
Remove-Item -Recurse -Force .\jogo-da-memoria -ErrorAction SilentlyContinue
```

macOS ou Linux

```
rm -rf jogo-da-memoria
```

Verifique o cache e recrie com SDK 54:

```
npm cache verify
npx create-expo-app@latest jogo-da-memoria --template default@sdk-54
```

8.2 node ou npm não é reconhecido

- Instale o Node.js LTS.
- Feche e abra o terminal.
- Reinicie o computador se necessário.
- Repita node --version e npm --version.

8.3 O projeto foi criado, mas o terminal está na pasta errada

Verifique o conteúdo da pasta:

Windows

```
dir
```

macOS ou Linux

```
ls
```

A pasta correta deve possuir package.json. Entre nela usando cd jogo-da-memoria.

8.4 Erro vermelho depois de editar o código

Leia a primeira mensagem do terminal ou da tela. Erros frequentes:

Mensagem ou sintoma	O que conferir
Unexpected token	Chave, parêntese, colchete, vírgula ou aspas ausentes.

Mensagem ou sintoma	O que conferir
Cannot find name	Nome digitado diferente da declaração.
Cannot find module	Import incorreto ou dependências ainda não instaladas.
Tela branca	Erro no terminal, return incompleto ou componente não fechado.
Text strings must be rendered	Texto colocado fora de um componente Text.

Estratégia de depuração

Compare somente o bloco que acabou de ser digitado. Não substitua o arquivo inteiro sem antes tentar localizar o caractere ou nome incorreto.

8.5 Alterações não aparecem

```
npx expo start --clear
```

- Confirme que o arquivo foi salvo.
- Confirme que o servidor está aberto na pasta correta.
- Pressione r no terminal para recarregar.
- Feche e reabra o aplicativo Expo Go.

8.6 O celular não abre o QR Code

- Confirme que computador e celular estão na mesma rede.
- Desative temporariamente VPNs que isolem a conexão.
- Verifique se o firewall permite o Node.js.
- Tente o navegador pressionando w.
- Tente o modo tunnel como alternativa.

```
npx expo start --tunnel
```

Observação

O modo tunnel depende de internet e pode iniciar mais lentamente.

8.7 A porta já está em uso

O Expo pode perguntar se deseja usar outra porta. Responda y. Também é possível fechar outros terminais que estejam executando projetos Expo.

8.8 Dependências incompatíveis

```
npx expo install --check
npx expo install --fix
npx expo-doctor@latest
```

Execute os comandos na raiz do projeto, onde está o arquivo package.json.

Parte 9 - Consolidação da aprendizagem

9.1 Perguntas para os alunos

1. Por que cada carta precisa de um id diferente, mesmo quando duas cartas possuem o mesmo emoji?

2. Qual estado guarda a primeira carta escolhida?
3. Por que o tabuleiro é bloqueado durante a comparação?
4. O que faz o método `map`?
5. O que faz o método `find`?
6. Qual parte inicia e encerra o cronômetro?
7. Como o programa identifica a vitória?
8. Qual propriedade muda quando um par é encontrado?

9.2 Pequenos desafios

Desafio	Conceito praticado
Trocar os emojis por frutas.	Arrays e conteúdo visual.
Alterar cada par para valer 20 pontos.	Constantes e cálculo.
Mudar as cores das cartas fechadas.	Estilos.
Alterar o texto da instrução.	JSX e Text.
Mostrar “Excelente!” se a vitória ocorrer em até 12 jogadas.	Condições.
Adicionar um botão “Fechar” ao modal.	Eventos e estado.

9.3 Checklist de encerramento da aula

- ☐ Todos os alunos conseguem iniciar o projeto com `npx expo start`.
- ☐ O projeto abre no Expo Go ou na web.
- ☐ As 16 cartas aparecem em quatro colunas.
- ☐ O cronômetro começa no primeiro toque.
- ☐ Os pares corretos recebem 10 pontos.
- ☐ As jogadas são contadas apenas após duas cartas.
- ☐ O botão Novo jogo reinicia todos os valores.
- ☐ O modal aparece ao atingir 80 pontos.
- ☐ Os alunos sabem onde estão os arquivos `_layout.tsx` e `index.tsx`.
- ☐ Os alunos conseguem explicar ao menos estado, evento e condição.

Apêndice A - Código completo de app/_layout.tsx

```
import { Stack } from 'expo-router';

export default function RootLayout() {
  return (
    <Stack
      screenOptions={{
        headerShown: false,
      }}
    />
  );
}
```

Apêndice B - Código completo de app/index.tsx

Como usar este apêndice

O código abaixo serve para conferência. Durante a aula, prefira construir por blocos. Copiar tudo de uma vez dificulta a compreensão e torna a correção de erros mais difícil.

```
import { useEffect, useState } from 'react';
import {
  FlatList,
  Modal,
  Pressable,
  ScrollView,
  StyleSheet,
  Text,
  useWindowDimensions,
  View,
} from 'react-native';
import { StatusBar } from 'expo-status-bar';

type MemoryCard = {
  id: number;
  emoji: string;
  isFlipped: boolean;
  isMatched: boolean;
};

const EMOJIS = [' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' '];
const POINTS_PER_PAIR = 10;
const MAX_SCORE = EMOJIS.length * POINTS_PER_PAIR;
const NUMBER_OF_COLUMNS = 4;
const CARD_GAP = 10;
const HORIZONTAL_PADDING = 20;

function createDeck(): MemoryCard[] {
  const duplicatedEmojis = [...EMOJIS, ...EMOJIS];

  const deck: MemoryCard[] = duplicatedEmojis.map((emoji, index) => ({
    id: index,
    emoji,
    isFlipped: false,
    isMatched: false,
  }));

  for (let index = deck.length - 1; index > 0; index--) {
    const randomIndex = Math.floor(Math.random() * (index + 1));
    const temporaryCard = deck[randomIndex];

    deck[index] = deck[randomIndex];
    deck[randomIndex] = temporaryCard;
  }

  return deck;
}

function formatTime(totalSeconds: number): string {
  const minutes = Math.floor(totalSeconds / 60);
  const seconds = totalSeconds % 60;
```



```

const formattedMinutes = String(minutes).padStart(2, '0');
const formattedSeconds = String(seconds).padStart(2, '0');

return `${formattedMinutes}:${formattedSeconds}`;
}

export default function Index() {
  const { width } = useWindowDimensions();

  const boardWidth = Math.min(
    width - HORIZONTAL_PADDING * 2,
    420,
  );

  const cardSize =
    (boardWidth - CARD_GAP * (NUMBER_OF_COLUMNS - 1)) /
    NUMBER_OF_COLUMNS;

  const [cards, setCards] = useState<MemoryCard[]>(() => createDeck());
  const [selectedCardId, setSelectedCardId] = useState<number | null>(null);
  const [boardLocked, setBoardLocked] = useState(false);
  const [score, setScore] = useState(0);
  const [attempts, setAttempts] = useState(0);
  const [seconds, setSeconds] = useState(0);
  const [timerRunning, setTimerRunning] = useState(false);
  const [showVictory, setShowVictory] = useState(false);

  useEffect(() => {
    if (!timerRunning) {
      return;
    }

    const intervalId = setInterval(() => {
      setSeconds((currentSeconds) => currentSeconds + 1);
    }, 1000);

    return () => {
      clearInterval(intervalId);
    };
  }, [timerRunning]);

  function startNewGame() {
    setCards(createDeck());
    setSelectedCardId(null);
    setBoardLocked(false);
    setScore(0);
    setAttempts(0);
    setSeconds(0);
    setTimerRunning(false);
    setShowVictory(false);
  }

  function handleCardPress(card: MemoryCard) {
    if (boardLocked || card.isFlipped || card.isMatched) {
      return;
    }

    if (!timerRunning && score < MAX_SCORE) {
      setTimerRunning(true);
    }

    setCards((currentCards) =>
      currentCards.map((currentCard) =>
        currentCard.id === card.id
          ? { ...currentCard, isFlipped: true }
          : currentCard,
      ),
    );

    if (selectedCardId === null) {
      setSelectedCardId(card.id);
      return;
    }

    const firstCard = cards.find(
      (currentCard) => currentCard.id === selectedCardId,
    );

    if (!firstCard) {
      setSelectedCardId(null);
      return;
    }
  }

```

```

    }

    setBoardLocked(true);
    setAttempts((currentAttempts) => currentAttempts + 1);

    const cardsMatch = firstCard.emoji === card.emoji;

    if (cardsMatch) {
      setTimeout(() => {
        setCards((currentCards) =>
          currentCards.map((currentCard) => {
            const belongsToPair =
              currentCard.id === firstCard.id ||
              currentCard.id === card.id;

            if (belongsToPair) {
              return {
                ...currentCard,
                isFlipped: true,
                isMatched: true,
              };
            }

            return currentCard;
          })
        ), 450);
      } else {
        setTimeout(() => {
          setCards((currentCards) =>
            currentCards.map((currentCard) => {
              const wasSelected =
                currentCard.id === firstCard.id ||
                currentCard.id === card.id;

              if (wasSelected && !currentCard.isMatched) {
                return { ...currentCard, isFlipped: false };
              }

              return currentCard;
            })
          ), 800);

          setSelectedCardId(null);
          setBoardLocked(false);
        }, 800);
      }
    }

    return (
      <View style={styles.screen}>
        <StatusBar style="dark" />

        <ScrollView
          contentContainerStyle={styles.screenContent}
          showsVerticalScrollIndicator={false}
        >
          <View style={styles.header}>
            <Text style={styles.title}>Jogo da Memória</Text>
            <Text style={styles.subtitle}>
              Encontre todos os pares de animais
            </Text>
          </View>

          <View style={styles.scoreboard}>
            <View style={styles.scoreItem}>
              <Text style={styles.scoreLabel}>Pontos</Text>
              <Text style={styles.scoreValue}>{score}</Text>
            </View>

            <View style={styles.scoreItem}>

```

```

      <Text style={styles.scoreLabel}>Jogadas</Text>
      <Text style={styles.scoreValue}>{attempts}</Text>
    </View>

    <View style={styles.scoreItem}>
      <Text style={styles.scoreLabel}>Tempo</Text>
      <Text style={styles.scoreValue}>{formatTime(seconds)}</Text>
    </View>
  </View>

  <View style={[styles.board, { width: boardWidth }]}>
    <FlatList
      data={cards}
      numColumns={NUMBER_OF_COLUMNS}
      keyExtractor={(item) => String(item.id)}
      scrollEnabled={false}
      extraData={` ${boardLocked} - ${cardSize}`}
      columnWrapperStyle={styles.cardRow}
      contentContainerStyle={styles.boardContent}
      renderItem={({ item }) => {
        const isVisible = item.isFlipped || item.isMatched;

        return (
          <Pressable
            accessibilityRole="button"
            accessibilityLabel={
              isVisible
                ? `Carta com o emoji ${item.emoji}`
                : 'Carta virada para baixo'
            }
            disabled={boardLocked || item.isFlipped || item.isMatched}
            onPress={() => handleCardPress(item)}
            style={({ pressed }) => [
              styles.card,
              { width: cardSize, height: cardSize },
              isVisible ? styles.cardVisible : styles.cardHidden,
              item.isMatched && styles.cardMatched,
              pressed && styles.cardPressed,
            ]}
          >
            <Text
              style={[
                styles.cardText,
                !isVisible && styles.hiddenCardText,
              ]}
              >
              {isVisible ? item.emoji : '?'}
            </Text>
          </Pressable>
        );
      }}
    />
  </View>

  <Pressable
    accessibilityRole="button"
    disabled={boardLocked}
    onPress={startNewGame}
    style={({ pressed }) => [
      styles.restartButton,
      pressed && styles.restartButtonPressed,
      boardLocked && styles.restartButtonDisabled,
    ]}
  >
    <Text style={styles.restartButtonText}>Novo jogo</Text>
  </Pressable>

  <Text style={styles.instructions}>
    Toque em duas cartas para procurar um par.
  </Text>
</ScrollView>

<Modal
  visible={showVictory}
  transparent
  animationType="fade"
  onRequestClose={() => setShowVictory(false)}
>
  <View style={styles.modalOverlay}>
    <View style={styles.modalContent}>
      <Text style={styles.modalEmoji}> </Text>
      <Text style={styles.modalTitle}>Parabéns!</Text>
    </View>
  </View>
</Modal>

```

```

    <Text style={styles.modalText}>
      Você encontrou todos os pares.
    </Text>

    <View style={styles.finalResult}>
      <Text style={styles.finalResultText}>Pontuação: {score}</Text>
      <Text style={styles.finalResultText}>Jogadas: {attempts}</Text>
      <Text style={styles.finalResultText}>
        Tempo: {formatTime(seconds)}
      </Text>
    </View>

    <Pressable
      accessibilityRole="button"
      onPress={startNewGame}
      style={({ pressed }) => [
        styles.modalButton,
        pressed && styles.restartButtonPressed,
      ]}
    >
      <Text style={styles.modalButtonText}>Jogar novamente</Text>
    </Pressable>
  </View>
</View>
</Modal>
</View>
);
}

const styles = StyleSheet.create({
  screen: {
    flex: 1,
    backgroundColor: '#f3f4f6',
  },
  screenContent: {
    alignItems: 'center',
    paddingTop: 50,
    paddingBottom: 28,
    paddingHorizontal: HORIZONTAL_PADDING,
  },
  header: {
    alignItems: 'center',
    marginBottom: 20,
  },
  title: {
    color: '#172554',
    fontSize: 30,
    fontWeight: '700',
    textAlign: 'center',
  },
  subtitle: {
    color: '#64748b',
    fontSize: 15,
    marginTop: 5,
    textAlign: 'center',
  },
  scoreboard: {
    width: '100%',
    maxWidth: 420,
    flexDirection: 'row',
    justifyContent: 'space-between',
    marginBottom: 22,
  },
  scoreItem: {
    width: '31%',
    alignItems: 'center',
    backgroundColor: 'ffffff',
    borderColor: '#dbeafe',
    borderRadius: 14,
    borderWidth: 1,
    paddingHorizontal: 5,
    paddingVertical: 12,
  },
  scoreLabel: {
    color: '#64748b',
    fontSize: 13,
  },
  scoreValue: {
    color: '#1e3a8a',
    fontSize: 20,
    fontWeight: '700',
    marginTop: 3,
  },
});

```

```

},
board: {
  alignSelf: 'center',
},
boardContent: {
  gap: CARD_GAP,
},
cardRow: {
  justifyContent: 'space-between',
},
card: {
  alignItems: 'center',
  justifyContent: 'center',
  borderRadius: 16,
  borderWidth: 2,
},
cardHidden: {
  backgroundColor: '#1e3a8a',
  borderColor: '#1e40af',
},
cardVisible: {
  backgroundColor: 'ffffff',
  borderColor: '#93c5fd',
},
cardMatched: {
  backgroundColor: '#dcfce7',
  borderColor: '#22c55e',
},
cardPressed: {
  opacity: 0.75,
},
cardText: {
  fontSize: 34,
},
hiddenCardText: {
  color: 'ffffff',
  fontSize: 28,
  fontWeight: '700',
},
restartButton: {
  width: '100%',
  maxWidth: 420,
  alignItems: 'center',
  backgroundColor: '#1e3a8a',
  borderRadius: 14,
  marginTop: 20,
  paddingVertical: 14,
},
restartButtonPressed: {
  opacity: 0.8,
},
restartButtonDisabled: {
  opacity: 0.5,
},
restartButtonText: {
  color: 'ffffff',
  fontSize: 16,
  fontWeight: '700',
},
instructions: {
  color: '#64748b',
  fontSize: 14,
  marginTop: 14,
  textAlign: 'center',
},
modalOverlay: {
  flex: 1,
  alignItems: 'center',
  justifyContent: 'center',
  backgroundColor: 'rgba(15, 23, 42, 0.65)',
  padding: 24,
},
modalContent: {
  width: '100%',
  maxWidth: 360,
  alignItems: 'center',
  backgroundColor: 'ffffff',
  borderRadius: 24,
  padding: 28,
},
modalEmoji: {
  fontSize: 58,

```

```

},
modalTitle: {
  color: '#172554',
  fontSize: 28,
  fontWeight: '700',
  marginTop: 8,
},
modalText: {
  color: '#64748b',
  fontSize: 16,
  marginTop: 8,
  textAlign: 'center',
},
},
finalResult: {
  width: '100%',
  backgroundColor: '#eff6ff',
  borderRadius: 14,
  marginVertical: 20,
  padding: 16,
},
finalResultText: {
  color: '#1e3a8a',
  fontSize: 16,
  marginVertical: 3,
  textAlign: 'center',
},
},
modalButton: {
  width: '100%',
  alignItems: 'center',
  backgroundColor: '#1e3a8a',
  borderRadius: 14,
  paddingVertical: 14,
},
modalButtonText: {
  color: 'ffffff',
  fontSize: 16,
  fontWeight: '700',
},
},
});

```

Referências técnicas consultadas

- [Expo Documentation - Create your first app](#)
- [Expo Documentation - create-expo-app](#)
- [Expo Documentation - Tools for development](#)
- [Expo Documentation - Expo SDK reference](#)
- [Expo Documentation - Upgrade Expo SDK](#)
- [Expo GitHub issue #48091 - npm 12 and create-expo-app JSON parsing](#)

Data de verificação

As orientações de versão e compatibilidade deste material foram verificadas em 27 de julho de 2026. Como ferramentas de desenvolvimento mudam com frequência, consulte a documentação oficial antes de reutilizar o tutorial em turmas futuras.